

DOKUMENTASI 2 BACKEND AGROSHOP

Nama : Muhamad Alfin Faizs

NPM : 23312174

Kelas : IF 23 E

Project : Agroshop

1. Tujuan

Pada tahap kedua, saya mulai mengembangkan fitur utama pada backend Agroshop. Fitur ini digunakan agar frontend dapat mengambil dan mengirim data melalui API.

2. Fitur yang Dikembangkan

- Auth untuk register, login, dan profile user.
- Products untuk menampilkan dan mengelola data produk.
- Categories untuk mengelola kategori produk.
- Banners untuk mengelola banner aplikasi.
- Cart untuk mengelola keranjang belanja user.

3. Endpoint API

Berikut beberapa endpoint utama yang sudah dibuat pada backend.

Modul	Endpoint	Fungsi
Auth	/auth/login, /auth/register	Login dan register user
Products	/products	Mengelola data produk
Categories	/categories	Mengelola kategori
Banners	/banners	Mengelola banner
Cart	/cart	Mengelola keranjang belanja

4. DTO Login dan Register

DTO digunakan untuk menampung data yang dikirim saat login dan register.

```
export class LoginDto {  
  email!: string;  
  password!: string;  
}
```

```
export class RegisterDto {
  name!: string;
  email!: string;
  password!: string;
  phone?: string;
  imageUrl?: string;
  address?: string;
}
```

5. Proses Register

Saat user register, password tidak langsung disimpan. Password akan di-hash menggunakan bcrypt agar lebih aman.

```
const existingUser = await this.prisma.user.findUnique({
  where: { email: registerDto.email },
});

if (existingUser) {
  throw new ConflictException('Email sudah terdaftar');
}

const saltRounds = 10;
const hashedPassword = await bcrypt.hash(registerDto.password, saltRounds);
```

6. Proses Login

Saat login berhasil, backend membuat token JWT. Token ini dipakai oleh frontend untuk mengakses endpoint yang membutuhkan login.

```
const user = await this.prisma.user.findUnique({
  where: { email: loginDto.email },
});

const isPasswordValid = await bcrypt.compare(
  loginDto.password,
  user.password,
);

const payload = { sub: user.id, email: user.email, role: user.role };
const token = this.jwtService.sign(payload);
```

7. JWT Strategy

JWT Strategy digunakan untuk membaca token dari header Authorization dan mengambil data user dari token tersebut.

```
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor() {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: 'Akudevalfin',
    });
  }

  async validate(payload: any) {
    return { userId: payload.sub, email: payload.email, role: payload.role };
  }
}
```

8. Proteksi Endpoint Admin

Beberapa endpoint hanya boleh diakses oleh admin, contohnya saat menambah produk.

```
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles('ADMIN')
@Post()
create(@Body() createProductDto: CreateProductDto) {
  return this.productsService.create(createProductDto);
}
```

9. Role Guard

RolesGuard digunakan untuk mengecek apakah role user sesuai dengan role yang dibutuhkan oleh endpoint.

```
const requiredRoles = this.reflector.get<string[]>(
  'roles',
  context.getHandler(),
);

const request = context.switchToHttp().getRequest();
const user = request.user;

if (!requiredRoles.includes(user.role)) {
  throw new ForbiddenException('Akses ditolak');
}
```

10. Logika Produk

Sebelum produk disimpan, backend mengecek apakah kategori produk tersedia di database.

```
const categoryExists = await this.prisma.category.findUnique({
  where: { id: createProductDto.categoryId },
});

if (!categoryExists) {
  throw new BadRequestException('Kategori tidak ditemukan');
}

await this.prisma.product.create({
  data: {
    name: createProductDto.name,
    price: createProductDto.price,
    stock: createProductDto.stock,
    categoryId: createProductDto.categoryId,
  },
});
```

11. Logika Category

Pada fitur kategori, backend juga menampilkan jumlah produk pada setiap kategori.

```
const rawData = await this.prisma.category.findMany({
  include: {
    _count: {
      select: { products: true },
    },
  },
});
```

12. Logika Cart

Pada cart, userId diambil dari token login. Jadi user yang sedang login akan otomatis menjadi pemilik cart.

```
create(@Req() req: any, @Body() createCartDto: CreateCartDto) {
  createCartDto.userId = req.user.userId;
  return this.cartService.create(createCartDto);
}
```

13. Menambah Quantity Cart

Jika produk yang sama sudah ada di cart, backend akan menambahkan quantity produk tersebut.

```
if (existingCartItem) {
  await this.prisma.cartItem.update({
    where: { id: existingCartItem.id },
    data: {
      quantity: existingCartItem.quantity + createCartDto.quantity,
    },
  });
}
```

14. Format Response API

Response dari backend dibuat dengan format success, message, metadata, dan data jika ada.

```
return {
  success: true,
  message: 'Berhasil mengambil semua data produk',
  metadata: {
    status: HttpStatus.OK,
    total_data: formattedData.length,
  },
  data: formattedData,
};
```

15. Hasil Tahap 2

- Fitur login dan register sudah berjalan.
- Token JWT sudah digunakan untuk melindungi endpoint tertentu.
- API produk, kategori, banner, user, dan cart sudah dibuat.
- Backend sudah dapat digunakan oleh frontend untuk mengambil dan mengirim data.

16. Rencana Tahap Selanjutnya

Pada dokumentasi 3, bagian yang dapat dibahas adalah penyempurnaan backend, seperti validasi DTO, pengaturan role yang lebih rapi, penggunaan environment variable, dan testing API.

17. Kesimpulan

Pada dokumentasi 2, backend Agrosshop sudah berkembang dari struktur awal menjadi API yang dapat digunakan oleh frontend. Fitur utama sudah tersedia, tetapi masih dapat disempurnakan pada tahap berikutnya.